

CS673 Software Engineering**Team 2 - HealthNest****Software Test Document**

Your project Logo
here if any

<u>Team Member</u>	<u>Role(s)</u>	<u>Signature</u>	<u>Date</u>
Deven Shah	Team Lead, Requirements Lead	<i>Deven Shah</i>	6/9/2026
Stephanie Wang	QA Lead, Design & Implementation Lead	<i>Stephanie Wang</i>	5/23/2026
Aaron Brown	Security Lead, Design & Implementation Lead	<i>Aaron Brown</i>	6/8/2026
Chamarr Auber	Team Lead, Configuration Lead	<i>Chamarr Auber</i>	5/24/2026

Revision history

<u>Version</u>	<u>Author</u>	<u>Date</u>	<u>Change</u>
2.0	Stephanie	6/4/2026	Updated STD for Iteration 2 testing scope, manual test cases, automated testing report, testing metrics
3.0	Stephanie	6/13/2026	Updated for Iteration 3 testing scope, manual test cases, automated testing report, testing metrics

[Testing Summary](#)

[Manual Tests Reports](#)

[Automated Testing Reports](#)

[Testing Metrics](#)

[References](#)

[Glossary](#)

● Testing Summary

In this section, you will summarize what was tested, who is involved in testing, when to test, testing techniques used, and testing result. You may have the following tests

- Unit Testing
- Integration testing
- System Testing
- Acceptance Testing
- Regression Testing

1.1 What Was Tested

The following features were in scope for last iteration:

- **SCRUM-7 User Login:** Sign-in, sign-up, sign-out, role selection, password visibility toggle, error handling, and partial biometric/passkey login support for Patient and Provider roles. This feature is covered by frontend Jest tests and manual test cases TC001–TC007, which remain part of the regression testing scope. Biometric authentication is currently covered at the frontend login-flow level, while full backend biometric integration is still in progress.
- **SCRUM-5 Appointment Scheduler:** Fully connected to the backend. Patients can view upcoming and past appointments, book new appointments by selecting a provider and available slot, reschedule, and cancel. Covered by frontend Jest unit tests, backend pytest tests for the appointments skill, and manual test cases TC008–TC010.
- **SCRUM-6 Secure Messaging:** Near-complete patient-provider messaging system supporting care-team contact retrieval, conversation history, message sending, realtime message delivery, unread message tracking, read-status synchronization, and message notifications. Covered by backend BDD and API testing of messaging services and workflows, as well as extensive manual end-to-end testing of patient-provider messaging scenarios, realtime updates, unread count tracking, and dashboard integration.
- **SCRUM-59 Patient Home Dashboard:** Patient dashboard navigation and shared top navigation were tested through TC015. The test confirmed that the patient can navigate to Dashboard, Appointments, My Care Team, Records,

Messages, and other patient-facing pages without breaking the signed-in session.

- **SCRUM-26 Doctor Home Dashboard:** Provider dashboard navigation and shared top navigation were tested through TC016. The test confirmed that the provider can navigate to Dashboard, Schedule, Patient Records, Messages, and related provider-facing pages while remaining signed in.
- **SCRUM-61 Lab Results:** Provider-facing lab result upload, review, edit, and release workflow was tested, along with the patient-facing released lab result view and flagged value display. This workflow is covered by TC011-TC014. TC031 also verifies Provider Searches and Opens Patient Record workflow.
- **SCRUM-39 Pulse AI:** Patient-facing AI assistant with streaming responses, citation chips, conversation history, and skill cards for appointments and lab results. Covered by frontend Jest unit tests and backend pytest tests spanning the guard module, orchestrator, skills registry, audit logging, pipeline, prompt construction, and schema validation.
- **SCRUM-58 DFA:** Visit Overview: Provider-facing AI Visit Summary / DFA Visit Overview behavior was tested through TC029. The test verified that the visit summary appears with the correct appointment and patient visit information.
- **SCRUM-80:** Sign Notes Encounter: Unsigned encounter workflow was tested through TC030. The test verified that unsigned encounters appear in the provider workflow and are removed from the unsigned list after signing.
- **SCRUM-75 My Care Team:** Patient care team retrieval and related actions were tested through TC023-TC025. These tests verified that the My Care Team page loads provider information, supports starting a message from a provider card, and supports booking an appointment from the care team page.
- **SCRUM-77 Provider Scheduling:** Provider schedule calendar behavior was tested through TC028. The test verified that the provider can view the schedule and switch between Day, Week, and Month views using date navigation controls.
- **SCRUM-56 / SCRUM-79 Biometric Login Setup:** Biometric/passkey setup was tested through the Account Settings workflow. TC020 verifies that a signed-in patient can select Enable Biometric Login and complete the browser passkey/biometric prompt on a supported device, or receive a clear unsupported/setup error message when biometric setup is unavailable. This confirms the frontend setup flow from Account Settings. Full backend WebAuthn verification and complete biometric login authentication remain limited and are not considered fully covered in this test cycle.

1.2 Who Is Involved in Testing

Feature Developers

- Write and update tests for the features they implement.
- Run local tests before opening or updating a pull request.
- Verify that their feature passes the required frontend, backend, and CI/CD checks.

Frontend Developers

- Maintain Jest and React Testing Library tests for frontend pages, components, helper functions, and API helper files.
- Verify user-facing workflows such as login, appointments, lab results, dashboards, and Pulse AI UI behavior.

Backend Developers

- Maintain pytest tests for backend logic and AI skill behavior.
- Verify backend workflows such as appointment skill logic, lab result skill logic, AI guard behavior, prompt construction, schema validation, and retrieval pipeline behavior.

Configuration Lead

- Reviews pull requests before merging.
- Maintains CI/CD pipeline behavior.
- Confirms that pull requests pass required automated checks before being merged into main

QA Lead

- Maintains the Software Test Document.
- Tracks manual and automated testing progress.
- Reviews whether test coverage matches the current implemented features.

All Team Members

- Participate in manual testing when needed.
- Review pull requests related to their feature areas.
- Report defects or missing coverage when issues are found.
- Help confirm that completed features meet the project requirements before release.

1.3 When Testing Occurs

Testing occurs throughout the Iteration 3 development and review process:

- **During feature development**
 - Developers run local tests while building or updating their assigned features.
 - Frontend developers run Jest tests for React components, pages, helper functions, and API helper files.
 - Backend developers run pytest tests for backend logic, Pulse AI skills, schemas, guard logic, and pipeline behavior.
- **Before opening a pull request**
 - Developers are expected to run relevant local tests before submitting their work for review.
 - This includes checking that the feature works locally and that existing tests are not broken.
- **During pull request review**
 - Each pull request is reviewed before merging into main.
 - Reviewers check that the feature matches the Jira ticket, does not introduce unrelated changes, and includes appropriate testing when applicable.
- **Through the CI/CD pipeline**
 - The CI/CD pipeline runs automatically on pull requests to main.

- The pipeline checks Jira naming conventions, frontend linting, frontend build status, Docker build status, and Docker Compose validation.
- Pull requests that fail required checks must be corrected before merging.
- **During regression testing**
 - Existing manual test cases and automated tests are reused to confirm that new changes do not break previously completed features.
- **After feature completion**
 - Manual testing is performed for completed user workflows such as login, appointment scheduling, lab result upload/release, and patient result viewing.
 - Any issues found during testing should be corrected before the feature is considered ready for release.

1.4 Testing Techniques Used

Unit Testing

- Frontend unit and component tests were written using Jest and React Testing Library.
- These tests verify individual frontend components, helper functions, API helper files, and page-level behavior.
- Unit testing was applied to:
 - User Login
 - Appointment Scheduler
 - Lab Results
 - Pulse AI frontend components
- Backend unit tests were written using pytest.
- Backend pytest testing was applied to:
 - Appointment AI skill logic
 - Lab results AI skill logic
 - Pulse AI guard logic
 - Prompt construction
 - Schema validation
 - Skills registry behavior
 - Audit logging
 - Retrieval pipeline behavior
 - Secure Messaging behavior

Integration Testing

- Integration testing was performed through connected workflows between the frontend, backend, and supporting services.
- The Appointment Scheduler was tested as a connected workflow where patients can view, book, reschedule, and cancel appointments.
- The Lab Results workflow was tested from provider upload and review through patient-facing result display.

System Testing

- System testing was performed by running the application locally and verifying complete user workflows.
- Tested system workflows included:

- Patient/provider login and sign-out
 - Patient appointment booking, viewing, and cancellation
 - Provider lab result upload, review, release, and patient viewing
 - Patient and provider top navigation
 - Patient and provider Account Settings
 - My Care Team page and care-team actions
 - Secure Messaging send/receive behavior and unread badge updates
 - Provider Schedule calendar views
 - DFA Visit Overview / AI Visit Summary display
 - Unsigned encounter signing workflow
 - Provider patient record search and detail viewing
- Manual test cases were used as acceptance tests for completed user-facing workflows.
 - TC001-TC007 covers the User Login workflow.
 - TC008-TC010 covers the Appointment Scheduler workflow.
 - TC011-TC014 covers the Lab Results workflow.
 - TC015-TC016 covers patient and provider top navigation.
 - TC017-TC022 covers Account Settings.
 - TC023-TC025 covers My Care Team and care-team actions.
 - TC026-TC027 covers Secure Messaging.
 - TC028 covers Provider Schedule views.
 - TC029 covers DFA Visit Overview / AI Visit Summary.
 - TC030 covers Unsigned Encounter signing.
 - TC031 covers Provider Searches and Opens Patient Record workflow.

These tests confirm that the completed features meet the expected user story behavior and that previously blocked appointment tests now pass in the latest code version.

Regression Testing

- Existing manual test cases and automated tests are reused to check that new changes do not break previously completed features.
- Iteration 1 login manual tests remain part of the Iteration 3 regression testing scope.
- Jest and pytest tests provide additional regression coverage for frontend components, backend AI skills, and shared helper logic.

- CI/CD checks also act as a basic regression safety net by requiring pull requests to pass linting, build checks, Docker checks, and naming convention checks before merging.

1.5 Testing Results

Testing results for Iteration 3 showed that the main completed workflows are covered through a combination of manual testing, frontend Jest tests, backend pytest tests, and CI/CD checks.

- **Manual testing results**
 - TC001–TC007 for User Login.
 - TC008–TC010 for Appointment Scheduler passed after retesting the previous appointment availability issue.
 - TC011–TC014 for Lab Results passed.
 - TC015–TC016 for patient and provider top navigation passed.
 - TC017–TC022 for Account Settings passed.
 - TC023–TC025 for My Care Team and care-team appointment/message actions passed.
 - TC026–TC027 for Secure Messaging passed.
 - TC028 for Provider Schedule calendar views passed.
 - TC029 for DFA Visit Overview / AI Visit Summary passed.
 - TC030 for Unsigned Encounter workflow passed.
 - TC031 for provider patient record search and detail viewing passed.
- **Frontend automated testing results**
 - Frontend Jest tests exist for User Login, Appointment Scheduler, Lab Results, and Pulse AI UI components.
 - These tests verify page rendering, user interactions, helper functions, API helper behavior, modal behavior, and error handling.
 - The frontend automated tests provide regression coverage for major user-facing workflows.
- **Backend automated testing results**
 - Backend pytest tests were added for Pulse AI and Secure Messaging backend behavior.
 - Pulse AI tests cover the appointments skill, lab results skill, guard logic, audit logging, AI client behavior, retrieval pipeline, prompt construction, schemas, skills registry, and source metadata. Backend pytest testing confirms that major Pulse AI backend helper logic and skill responses behave as expected.
 - Secure Messaging backend testing was implemented using behavior-driven development (BDD) scenarios that validate core messaging workflows. These tests verify successful message creation, inbox retrieval, authentication enforcement, and patient-provider communication behavior. The scenarios confirm that authenticated patients can send messages, view received messages through their inbox, and that unauthenticated users are prevented from accessing protected messaging functionality.
- **CI/CD results**

- Pull requests are checked through the CI/CD pipeline before merging.
- Current CI/CD checks include Jira naming validation, frontend linting, frontend build, Docker build, and Docker Compose validation.
- Pull requests that fail required checks must be corrected before merging.

● Manual Testing Report

In this section, you will give a detailed description of each manual test case performed and the result. If this is a previous You shall list what are existing tests developed in the previous semester and what are new tests developed currently.

Here is a sample template that can be used for each test case. For system tests or acceptance tests, you may also include some screenshots.

- Test case ID, name
- New or old:
- Test items: (what do you test)
- Test priority (high/medium/low)
- Dependencies (to other test case/requirement if any):
- Preconditions: (if any)
- input data:
- Test steps:
- Postconditions:
- Expected output:
- Actual output:
- Pass or Fail:
- Bug id/link: (this should link to your github issue id)
- Additional notes:

(You can use an additional spreadsheet for this section as well)

Test Case ID	TC01
Name	Patient Sign-Up with Valid Credentials
New or Old	New
Test Items	SCRUM-7 User Login — Sign-up flow for Patient role
Test Priority	High
Dependencies	Supabase authentication service must be running
Preconditions	Email address not previously registered in the system
Input Data	Full Name, MRN: MRN-00123456, Date of Birth: 05/22/2026, Sex at Birth: Prefer not to say, Email: BU email, Password: 6+ character password
Test Steps	1. Click 'Create an account as Patient'

	2. Select Patient role 3. Fill in all required fields 4. Click 'Create patient account'
Postconditions	New user account created and stored in Supabase database
Expected Output	Success message
Actual Output	Green success message displayed: 'Account created. Check your email to confirm before signing in.'
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	User record appeared in Supabase authentication dashboard after sign-up

Test Case ID	TC02
Name	Patient Sign-In with Valid Credentials
New or Old	New
Test Items	SCRUM-7 User Login — Sign-in flow for Patient role
Test Priority	High
Dependencies	TC01 must pass; user account must exist and email must be confirmed
Preconditions	User account created and email confirmation link clicked
Input Data	Email: BU email, Password: valid password, Role: Patient
Test Steps	1. Select Patient role 2. Enter valid email and password 3. Click 'Sign in as Patient'
Postconditions	User is authenticated and session is created in Supabase
Expected Output	User is successfully signed in and redirected to the Patient Dashboard
Actual Output	User signed in successfully and redirected to Patient Dashboard
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	None

Test Case ID	TC03
Name	Role-Based Routing — Patient Redirected to Patient Dashboard

New or Old	New
Test Items	SCRUM-7 User Login — Role-based routing after authentication
Test Priority	High
Dependencies	TC02 must pass
Preconditions	User signed in as Patient role
Input Data	Patient role selected at login
Test Steps	1. Sign in as Patient 2. Observe which page loads after successful login
Postconditions	User lands on Patient Dashboard
Expected Output	Application automatically routes Patient user to the Patient Dashboard after login
Actual Output	Patient Dashboard loaded immediately after sign-in with no manual navigation required
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	None

Test Case ID	TC04
Name	Sign-In with Wrong Password
New or Old	New
Test Items	SCRUM-7 User Login — Error handling for invalid credentials
Test Priority	High
Dependencies	TC01 must pass; user account must exist
Preconditions	Valid user account exists in the system
Input Data	Email: BU Email Password: incorrect password
Test Steps	1. Select Patient role 2. Enter valid email and an incorrect password 3. Click 'Sign in as Patient'
Postconditions	User remains on login page; no session created
Expected Output	Error message displayed indicating invalid credentials
Actual Output	Red error message displayed: 'Invalid login credentials'
Pass or Fail	PASS
Bug ID / Link	N/A

Additional Notes	Error message is clear
-------------------------	------------------------

Test Case ID	TC05
Name	Sign-In with Empty Fields
New or Old	New
Test Items	SCRUM-7 User Login — Empty field validation on sign-in
Test Priority	Medium
Dependencies	None
Preconditions	User is on the login page
Input Data	Email: empty, Password: empty
Test Steps	1. Select Patient role 2. Leave email and password fields empty 3. Click 'Sign in as Patient'
Postconditions	Form does not submit; user remains on login page
Expected Output	Submission is prevented and user is prompted to fill in required fields
Actual Output	Browser native autocomplete dropdown appeared; form submission was prevented by browser-level validation
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	Validation is currently handled by browser native behavior. A future improvement could be to add a custom app-level error message such as 'Please enter your email and password' for a more consistent user experience.

Test Case ID	TC06
Name	Sign-Up with Password Too Short
New or Old	New
Test Items	SCRUM-7 User Login — Password length validation on sign-up
Test Priority	High
Dependencies	None
Preconditions	User is on the sign-up page
Input Data	Email: BU Email, Password: 'ab' (2 characters)

Test Steps	1. Click 'Create an account as Patient' 2. Fill in name, MRN, and email fields 3. Enter a password with only 2 characters 4. Click 'Create patient account'
Postconditions	Account is not created; user remains on sign-up page
Expected Output	Error message displayed indicating password is too short (minimum 6 characters)
Actual Output	Error message displayed: 'Please use at least 6 characters (you are currently using 2 characters).'
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	Password minimum length of 6 characters is enforced correctly by the application

Test Case ID	TC07
Name	Patient Sign-Out
New or Old	New
Test Items	SCRUM-7 User Login — Sign out flow for Patient role
Test Priority	High
Dependencies	TC002 must pass; user must be signed in
Preconditions	User is signed in as Patient and on the Patient Dashboard
Input Data	Authenticated Patient session
Test Steps	1. Sign in as Patient 2. On the Patient Dashboard, click the Sign Out button 3. Observe which page loads
Postconditions	User session is terminated; user is logged out of the system
Expected Output	User is signed out and redirected back to the Login page
Actual Output	User was signed out and redirected directly back to the Login page
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	Sign-out completes immediately with no delay or confirmation prompt

Test Case ID	TC08
Name	Patient Books and Appointment

New or Old	Updated
Test Items	SCRUM-5 Appointment Scheduler – Booking flow for Patient role
Test Priority	High
Dependencies	TC002 must pass; at least one provider with available slots must exist
Preconditions	Patient is signed in and on the Booking page
Input Data	Authenticated Patient session; provider selected from the booking page
Test Steps	1. Sign in as a Patient. 2. Navigate to the Appointments page. 3. Click the Book Appointment button. 4. Select a provider from the available provider list. 5. Select an available date and time slot. 6. Confirm the appointment booking. 7. Return to the Appointments page and check the Upcoming tab.
Postconditions	N/A
Expected Output	Available time slots displayed and appointment created successfully
Actual Output	The patient was able to select a provider, view available slots, choose a time, and successfully create an appointment. The new appointment appeared in the Upcoming tab after booking.
Pass or Fail	Pass
Bug ID / Link	SCRUM-72
Additional Notes	Additional Notes: This test was updated from the previous Iteration 2 result where the appointment availability endpoint returned a backend error. The latest version should be manually retested to confirm the issue is resolved.

Test Case ID	TC09
Name	Patient Views Upcoming and Past Appointments
New or Old	Updated
Test Items	SCRUM-5 Appointment Scheduler – Booking flow for Patient role
Test Priority	High
Dependencies	TC008 must pass; at least one scheduled appointment
Preconditions	Patient is signed in with at least one scheduled appointment
Input Data	Authenticated Patient session with at least one scheduled appointment
Test Steps	1. Sign in as Patient and navigate to the Appointments page 2. Observe the default tab 3. Click the Past tab 4. Click the All tab
Postconditions	Appointments are displayed and filtered correctly by tab

Expected Output	Upcoming tab shows only future appointments; Past tab shows only completed appointment; All tab shows both
Actual Output	Upcoming, Past, and All tabs displayed the correct appointment groups. The appointment created in TC08 appeared correctly in the Upcoming tab.
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	This test was previously blocked because TC08 failed. It should now be rerun after confirming appointment booking works in the latest code.

Test Case ID	TC10
Name	Patient Cancels an Appointment
New or Old	New
Test Items	SCRUM-5 Appointment Scheduler – Appointment cancellation for Patient role
Test Priority	High
Dependencies	TC008 must pass; at least one scheduled appointment
Preconditions	Patient is signed in with at least one scheduled appointment
Input Data	Authenticated Patient session with at least one scheduled appointment
Test Steps	1. Sign in as Patient and navigate to the Appointments page 2. Find a scheduled appointment 3. Click the Cancel button 4. Confirm cancellation in the confirmation modal 5. Observe the appointment list
Postconditions	Appointments is cancelled in the database and removed from the upcoming appointments list
Expected Output	Appointment status changes to cancelled and it is removed from the Upcoming tab
Actual Output	The patient cancelled the appointment successfully. The appointment was removed from the Upcoming tab or no longer displayed as an active upcoming appointment
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	This test was previously blocked because appointment booking failed in TC08. It should now be rerun after confirming that TC08 passes.

Test Case ID	TC11
Name	Provider Uploads a Lab Result File

New or Old	New
Test Items	SCRUM-61 Lab Results — File upload flow for Provider role
Test Priority	High
Dependencies	User must be signed in as Provider; a valid HLT lab result file must be available
Preconditions	Provider is signed in and on the Lab Results page
Input Data	Authenticated Provide session; sample HL7 file
Test Steps	1. Sign in as Provider and navigate to the Lab Results page 2. Click Browse files or drag and drop a lab result file onto the upload area 3. Select a patient from the patient typeahead search 4. Click upload & review
Postconditions	Lab result file is uploaded and parsed; the provider is taken to the review page
Expected Output	File uploads successfully and the lab result appears in the pending results list with status uploaded
Actual Output	File uploaded successfully. Parsed entries appeared in the review form.
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	

Test Case ID	TC12
Name	Provider Reviews and Releases a Lab Result
New or Old	New
Test Items	SCRUM-61 Lab Results — Review and release flow for Provider role
Test Priority	High
Dependencies	TC011 must pass; an uploaded lab result must exist with status uploaded
Preconditions	Provider is signed in; an uploaded lab result is pending review
Input Data	Authenticated Provide session; uploaded lab result pending review
Test Steps	1. Sign in as Provider and navigate to the Lab Results page 2. Click “Review & Release” on a pending lab result 3. Review the parsed entries and verify the lab name and patient 4. Click “Release to patient” 5. Confirm the release action
Postconditions	Lab result status changes to “released” and it becomes visible in the patient-facing lab results view
Expected Output	Result is released and removed from the pending review list

Actual Output	The result was released successfully. If no longer appeared in the pending list and became visible in the patient view.
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	Provider can edit any field in the review form before releasing

Test Case ID	TC13
Name	Patient Views a Released Lab Result
New or Old	New
Test Items	SCRUM-61 Lab Results — Patient-facing lab result view
Test Priority	High
Dependencies	TC012 must pass; at least one released lab result must exist for the patient
Preconditions	Provider is signed in; a released lab result exists for this patient
Input Data	Authenticated Patient session; released lab result in the system
Test Steps	1. Sign in as Patient and navigate to the Lab Results page 2. Find a released lab result 3. Click "View results" 4. Review the entries table
Postconditions	Patient can see the lab result with test names, values, reference ranges, and flagged values highlighted
Expected Output	Lab result entries display correctly with abnormal values flagged and an explanation messages shown
Actual Output	Lab result entries display correctly
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	Flagged values are visually distinct from normal values

Test Case ID	TC14
Name	Patient Views Lab Results with Flagged Values
New or Old	New
Test Items	SCRUM-61 Lab Results — Flagged value display in provider review
Test Priority	High

Dependencies	TC011 must pass; an uploaded lab result with at least one abnormal value must exist
Preconditions	Provider is signed in; a lab result with flagged entries has been uploaded
Input Data	Authenticated Provider session; lab result with abnormal entries
Test Steps	1. Sign in as Provider and navigate to the Lab Results page 2. Click "Review & Release" on a result with flagged values 3. Observe the entire table
Postconditions	Flagged entries are visually distinguished from normal ones in the review form
Expected Output	Abnormal entries are highlighted and the flagged count is shown in the result summary
Actual Output	Abnormal entries were highlighted in the review table. The result summary showed the correct flagged count.
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	Critical values are visually distinct from regular high/low flags

Test Case ID	TC15
Name	Patient Top Navigation Works Across Pages
New or Old	New
Test Items	SCRUM-59 Patient Dashboard — Shared TopNav / patient navigation
Test Priority	High
Dependencies	TC002 must pass; patient user must be signed in
Preconditions	Patient is signed in and on Patient Dashboard
Input Data	Valid patient account credentials
Test Steps	1. Sign in as Patient 2. Click Dashboard, Appointments, My Care Team, Records, Messages, and Pulse AI from the top nav 3. Return to Dashboard using the HealthNest logo
Postconditions	Patient is signed in and on Patient Dashboard
Expected Output	Each navigation link opens the correct patient page without breaking the session or showing a blank page
Actual Output	Each navigation link connects to the correct page, showing the correct content
Pass or Fail	PASS
Bug ID / Link	
Additional Notes	Verify all navigation links are visible and functional

Test Case ID	TC16
Name	Provider Top Navigation Works Across Pages
New or Old	New
Test Items	SCRUM-26 Doctor Dashboard — Shared TopNav / provider navigation
Test Priority	High
Dependencies	Provider sign-in must work; provider user must be signed in
Preconditions	Provider is signed in and on Doctor Dashboard
Input Data	Valid provider account credentials
Test Steps	1. Sign in as Provider 2. Click Dashboard, Schedule, Patient Records, Messages, and Pulse AI from the top nav 3. Return to Dashboard using the HealthNest logo
Postconditions	Provider remains signed in and returns to Dashboard
Expected Output	Each navigation link opens the correct provider page and keeps the provider signed in
Actual Output	Each navigation link connects to the correct page, showing the correct content
Pass or Fail	PASS
Bug ID / Link	
Additional Notes	Verify all provider navigation links are visible and functional

Test Case ID	TC17
Name	Patient Opens Account Settings
New or Old	New
Test Items	SCRUM-79 Account Settings — Account Settings navigation for Patient role
Test Priority	High
Dependencies	TC002 and TC015 must pass
Preconditions	Patient is signed in
Input Data	Valid patient account
Test Steps	1. Open the user dropdown in the top nav 2. Click Account Settings 3. Click Back to Dashboard
Postconditions	Patient returns to Dashboard after viewing settings

Expected Output	Patient Account Settings page opens successfully and returns to Patient Dashboard correctly
Actual Output	Patient Account Settings page opens and be able to return to Patient Dashboard
Pass or Fail	PASS
Bug ID / Link	#63
Additional Notes	Confirm all settings sections load correctly

Test Case ID	TC18
Name	Patient Edits Display Name
New or Old	New
Test Items	SCRUM-79 Account Settings — Patient Account Settings profile edit
Test Priority	High
Dependencies	TC017 must pass
Preconditions	Patient is signed in and on Account Settings page
Input Data	New patient display name
Test Steps	1. Click Edit next to Name 2. Change the patient name 3. Click Save 4. Leave Account Settings and reopen it
Postconditions	Updated name remains saved in profile
Expected Output	Updated patient name is saved and displayed correctly after returning to Account Settings
Actual Output	Name saved and displayed correctly to both setting page and dashboard
Pass or Fail	PASS
Bug ID / Link	#63
Additional Notes	Use a unique test name for validation

Test Case ID	TC19
Name	Patient Password Validation
New or Old	New
Test Items	SCRUM-79 Account Settings — Patient Account Settings password change validation

Test Priority	High
Dependencies	TC017 must pass
Preconditions	Patient is signed in and on Account Settings page
Input Data	Blank fields, mismatched passwords, password under 6 characters
Test Steps	1. Click Change Password 2. Leave one required field empty and submit 3. Try mismatched new passwords 4. Try a password shorter than 6 characters
Postconditions	Password remains unchanged
Expected Output	System prevents invalid password update and displays a clear validation message
Actual Output	System shows a clear validation message when inputting invalid password
Pass or Fail	PASS
Bug ID / Link	
Additional Notes	Verify all validation messages are user-friendly

Test Case ID	TC20
Name	Patient Enables Biometric Login
New or Old	New
Test Items	SCRUM-79 Account Settings — Patient Account Settings biometric/passkey setup
Test Priority	Medium
Dependencies	TC017 must pass; browser/device must support passkey or biometric prompt
Preconditions	Patient is signed in and on Account Settings page
Input Data	Supported browser/device with biometric capability
Test Steps	1. Click Enable Biometric Login 2. Complete the browser passkey/biometric prompt
Postconditions	Biometric login setting is enabled or setup is cancelled gracefully
Expected Output	Biometric setup succeeds, or the system shows a clear unsupported/setup error message
Actual Output	User is able to enable biometric function
Pass or Fail	PASS
Bug ID / Link	#54
Additional Notes	Test on both supported and unsupported devices if possible

Test Case ID	TC21
Name	Provider Opens Account Settings
New or Old	New
Test Items	SCRUM-79 Account Settings — Account Settings navigation for Provider role
Test Priority	High
Dependencies	TC016 must pass; provider user must be signed in
Preconditions	Provider is signed in and on Doctor Dashboard
Input Data	Valid provider account
Test Steps	1. Open the user dropdown in the top nav 2. Click Account Settings 3. Click Back to Dashboard
Postconditions	Provider returns to Dashboard after viewing settings
Expected Output	Provider Account Settings page opens successfully and returns to Doctor Dashboard correctly
Actual Output	User is able to open account setting and return to dashboard
Pass or Fail	PASS
Bug ID / Link	#54
Additional Notes	Confirm all provider settings sections load correctly

Test Case ID	TC22
Name	Provider Edits Specialty
New or Old	New
Test Items	SCRUM-79 Account Settings — Provider Account Settings specialty edit
Test Priority	Medium
Dependencies	TC021 must pass
Preconditions	Provider is signed in and on Account Settings page
Input Data	Updated specialty value
Test Steps	1. Click Edit next to Specialty 2. Enter or update the specialty value 3. Click Save 4. Leave Account Settings and reopen it
Postconditions	Updated specialty remains saved in profile

Expected Output	Specialty field is editable, saves successfully, and displays the updated specialty
Actual Output	Specialty is able to edit, save and display
Pass or Fail	PASS
Bug ID / Link	
Additional Notes	Verify specialty persists after page refresh

Test Case ID	TC23
Name	My Care Team Page Loads
New or Old	New
Test Items	SCRUM-75 My Care Team — Care Team retrieval
Test Priority	High
Dependencies	TC002 and TC015 must pass; care team data should exist or empty state should be supported
Preconditions	Patient is signed in
Input Data	Existing care team data or empty-state scenario
Test Steps	1. Navigate to My Care Team 2. Observe provider cards or empty-state message
Postconditions	Care Team page remains accessible
Expected Output	Care team providers display with name, specialty, Message button, and Book button, or a clear empty-state message appears
Actual Output	Care team providers display with name, specialty (optional), message button, and book button.
Pass or Fail	PASS
Bug ID / Link	
Additional Notes	

Test Case ID	TC24
Name	Patient Starts Message from Care Team
New or Old	New
Test Items	SCRUM-75 My Care Team — Care Team to Messages integration

Test Priority	High
Dependencies	TC023 must pass; patient must have at least one care team provider
Preconditions	Patient is signed in and on My Care Team page
Input Data	Existing provider in care team
Test Steps	1. Click Message on a provider card
Postconditions	Messages page opens with selected provider conversation
Expected Output	Application opens the Messages page with the selected provider conversation ready
Actual Output	Messages page open and able to send to the selected provider
Pass or Fail	PASS
Bug ID / Link	
Additional Notes	The provider is able to review the message

Test Case ID	TC25
Name	Patient Books Appointment from Care Team
New or Old	New
Test Items	SCRUM-75 My Care Team — Care Team to Appointment booking integration
Test Priority	High
Dependencies	TC023 must pass; appointment availability endpoint must work; provider must have available slots
Preconditions	Patient is signed in and on My Care Team page
Input Data	Available appointment date and time slot
Test Steps	1. Click Book on a provider card 2. Select an available date and time slot 3. Confirm booking
Postconditions	Appointment is created and visible in appointment records
Expected Output	Appointment modal opens for the selected provider and appointment is created successfully
Actual Output	Appointment modal opens and able to select provider, time and schedule
Pass or Fail	PASS
Bug ID / Link	
Additional Notes	Verify appointment details are accurate

Test Case ID	TC26
Name	Secure Messaging Send and Receive
New or Old	New
Test Items	SCRUM-54 Secure Messaging — end-to-end patient-provider flow
Test Priority	High
Dependencies	TC024 must pass; patient and provider accounts must exist; care team contact must exist
Preconditions	Patient and provider can both sign in
Input Data	Test message content
Test Steps	1. Patient opens Messages 2. Selects provider conversation 3. Sends a message 4. Provider signs in and opens Messages
Postconditions	Message is stored in conversation history
Expected Output	Message appears in the thread for both users with correct sender, timestamp, and conversation history
Actual Output	Message appears with correct sender, timestamp, and in conversation history
Pass or Fail	PASS
Bug ID / Link	
Additional Notes	Verify timestamps and sender information

Test Case ID	TC27
Name	Secure Messaging Unread Badge Updates
New or Old	New
Test Items	SCRUM-54 Secure Messaging — Messaging unread count and read status
Test Priority	Medium
Dependencies	TC026 must pass
Preconditions	A message has been sent from one role to another
Input Data	Existing unread message
Test Steps	1. Sign in as the message recipient 2. Observe the Messages unread badge

	3. Open the conversation thread 4. Return to top navigation
Postconditions	Message status changes from unread to read
Expected Output	Unread badge increments before opening the message and clears or updates after the message thread is read
Actual Output	The number pops out when there is unread messages and clears or updates after message is read
Pass or Fail	PASS
Bug ID / Link	
Additional Notes	Verify badge updates in real time if supported

Test Case ID	TC28
Name	Provider Schedule Day/Week/Month Views
New or Old	New
Test Items	SCRUM-77 Provider Scheduling — Provider Schedule calendar views
Test Priority	High
Dependencies	TC016 must pass
Preconditions	Provider is signed in and on Schedule page
Input Data	Existing schedule data
Test Steps	1. Navigate to Schedule 2. Switch between Day, Week, and Month views 3. Use previous, next, and today controls
Postconditions	Selected calendar view remains active
Expected Output	Schedule renders correctly in each view and date controls update the calendar period correctly
Actual Output	Schedule renders correctly in each view and date controls update the calendar period correctly
Pass or Fail	PASS
Bug ID / Link	#56
Additional Notes	Verify layout consistency across views

Test Case ID	TC29
Name	AI Visit Summary Appears on Today's Schedule
New or Old	New
Test Items	SCRUM-58 DFA Visit Overview — AI Visit Summary Overview and provider schedule integration
Test Priority	High
Dependencies	TC025 must pass; completed appointment and AI summary generation must be available
Preconditions	Provider is signed in and has a completed patient visit
Input Data	Completed appointment with generated AI summary
Test Steps	1. Complete a patient appointment 2. Generate or wait for AI Visit Summary 3. Open Provider Dashboard or Today's Schedule 4. Select the appointment 5. Open the AI Visit Summary Overview
Postconditions	AI summary is associated with the completed encounter
Expected Output	Appointment appears on Today's Schedule and AI Visit Summary Overview displays the correct patient, visit details, notes, and summary content
Actual Output	Both today's schedules and AI visit summary overview has the correct appointment information
Pass or Fail	PASS
Bug ID / Link	
Additional Notes	Verify summary accuracy and linkage to encounter record

Test Case ID	TC30
Name	Unsigned Encounter Appears in Provider Workflow
New or Old	New
Test Items	SCRUM-80 Sign Notes Encounter — Unsigned encounter management
Test Priority	High
Dependencies	TC029 must pass
Preconditions	Provider has at least one completed encounter not yet signed
Input Data	Completed encounter awaiting signature

Test Steps	1. Complete a patient visit 2. Leave encounter unsigned 3. Navigate to Unsigned Encounters or Dashboard notification area 4. Open the encounter 5. Review and sign encounter
Postconditions	Encounter status changes from unsigned to signed
Expected Output	Unsigned encounter is visible to the provider, opens correctly, and is removed from the unsigned list after signing
Actual Output	Unsigned encounter is visible to the provider, opens correctly, and is removed from the unsigned list after signing
Pass or Fail	PASS
Bug ID / Link	#58
Additional Notes	Verify encounter status updates throughout the system

Test Case ID	TC31
Name	Provider Searches and Opens Patient Record
New or Old	New
Test Items	Patient Records — provider patient record search and detail view
Test Priority	High
Dependencies	TC016 must pass; provider account and at least one patient record must exist
Preconditions	Provider is signed in and on the Patient Records page
Input Data	Existing patient name or MRN
Test Steps	1. Sign in as Provider 2. Navigate to Patient Records 3. Search for an existing patient by name or MRN 4. Open the matching patient record 5. Review the patient details, recent appointments, lab results, or encounter information shown in the record
Postconditions	Provider remains on the selected patient record page
Expected Output	Provider can search for a patient, open the correct patient record, and view the correct patient information without seeing unrelated patient data
Actual Output	Provider can search for a patient, open the correct patient record, and view the correct patient information without seeing unrelated patient data
Pass or Fail	PASS
Bug ID / Link	#57
Additional Notes	

● Automated Testing Report

Automated testing was expanded during Iteration 3 to improve regression coverage for the main implemented features. The project now includes both frontend automated tests and backend automated tests.

Frontend Automated Testing

Front-end automated tests are written using Jest and React Testing Library. The frontend test files are located in:

frontend/src/__tests__/

The frontend automated tests cover:

SCRUM-7 User Login

- Login form rendering
- Patient and Provider role selection
- Sign-in and sign-up behavior
- Sign-out behavior
- Password visibility toggle
- Error handling
- Partial biometric/passkey login flow

SCRUM-5 Appointment Scheduler

- Appointment API helper behavior
- Appointments page rendering
- Appointment card behavior
- Booking page behavior
- Appointment modal behavior
- Confirmation modal behavior

SCRUM-61 Lab Results

- Lab result helper functions
- Lab results page rendering
- Lab result detail view
- Lab result review workflow
- Lab result upload modal
- Lab results card behavior
- Patient lab results page behavior

SCRUM-39 Pulse AI

- Pulse API helper behavior
- Pulse provider state
- Citation chips
- Composer
- Conversation sidebar

- Conversation thread
- Message bubble
- Skill chips
- Skill response card

These frontend tests verify that major user-facing pages and components render correctly, respond to expected user interactions, handle modal workflows, and preserve expected UI behavior during regression testing.

Backend Automated Testing

Backend automated tests are written using pytest. The backend ai test files are located in `backend/ai/tests/` and `backend/tests`

The backend automated tests cover:

- Appointment AI skill behavior
- Lab results AI skill behavior
- AI assistant behavior
- Audit logging
- AI client configuration
- Emergency guard logic
- Retrieval orchestration and patient tenancy
- Retrieval pipeline behavior
- Prompt construction
- Request and response schema validation
- Skills registry behavior
- Source metadata behavior
- Secure Messaging API and workflow behavior, including message creation, inbox retrieval, authentication enforcement, and patient-provider communication scenarios

CI/CD Automated Checks

The project also uses GitHub Actions CI/CD checks on pull requests. These checks help prevent broken or incorrectly formatted code from being merged into main.

The CI/CD pipeline includes:

- Jira key validation
- Frontend ESLint check
- Frontend build check
- Docker build check
- Docker Compose validation

Jest and pytest do not currently run as part of the CI/CD pipeline. At this stage, frontend Jest tests and backend pytest tests are run locally only. Adding Jest and pytest as required CI/CD steps is planned for a future iteration.

Automated Testing Coverage Summary

The automated test suite supports the manual testing process by checking lower-level frontend and backend behavior, while the manual test suite verifies full user workflows from the Patient and Provider perspectives. Automated testing covers login, appointment components and helpers, lab result components and helpers, Pulse AI frontend and

backend behavior, backend skill logic, schema validation, audit logging, retrieval behavior, and secure messaging backend workflows.

The final manual test cycle expands workflow-level coverage to TC001-TC031. These manual tests cover areas that are harder to fully validate through unit tests alone, including top navigation, Account Settings, My Care Team, Secure Messaging user flow, Provider Schedule, DFA Visit Overview, Unsigned Encounters, and Patient Records search/detail viewing.

Current Automated Testing Limitations

Although the combined testing approach provides broad coverage across the implemented system, the following items remain future automation improvements:

- Jest and pytest are currently run locally and are not yet required CI/CD steps.
- Full backend biometric/WebAuthn verification testing remains limited. Current testing covers the frontend biometric/passkey setup flow, but full backend verification is not fully automated yet.
- Some newer workflows, including Account Settings, My Care Team, Provider Schedule, DFA Visit Overview, Unsigned Encounter signing, and Patient Records search/detail viewing, are currently covered through manual workflow testing and can be expanded with dedicated frontend unit tests in a future iteration.
- The appointment availability issue identified in the previous test cycle was manually retested. TC008, TC009, and TC010 now pass in the latest version, so the previous failed/blocked appointment status has been resolved.

Overall, the project now has testing coverage at multiple levels: frontend component and helper testing, backend pytest testing, secure messaging workflow testing, CI/CD validation, and completed manual acceptance testing for the major Patient and Provider workflows.

● Testing Metrics

In this section, you shall report any metrics used for the evaluation, e.g. # of test cases, test coverage, defects rate, etc.

Metric	Value	Notes
Total manual test cases	31	TC01-TC31
Existing regression manual tests cases	14	TC001-TC014 from the previous STD scope were reused for regression testing
New Manual test cases added	17	TC015-TC031 were added for the final testing cycle
Manual test passed	31	All completed manual test cases passed

Manual tests failed	0	No manual test cases failed in the final test cycle
Manual tests blocked	0	No test cases were blocked after retesting the appointment workflow
Test pass rate	100%	31 passed out of 31 total manual test cases
Previously blocked tests resolved	2	TC009 and TC010 were previously blocked by TC008 and now pass
New defects found during final manual testing	0	No new defects were found during the completed TC001-TC031 manual test cycle
Defect resolution status	Resolved	The previously reported appointment availability issue no longer blocks appointment testing
Frontend automated testing	Present	Jest and React testing library tests exist for login, appointments, lab results, and Pulse AI frontend behavior
Frontend automated tests passed	259/259	All frontend automated tests passed
Frontend statement coverage	47.82%	Measured from Jest coverage report
Frontend line coverage	50.63%	Measured from Jest coverage report
Backend automated testing	Present	Pytest tests exist for Pulse AI backend behavior, appointment/lab result skill behavior, and Secure Messaging backend workflows
Backend automated tests passed	227/227	All backend pytest tests passed
Backend code coverage	63%	Measured from pytest-cov coverage report
CI/CD checks passed	Present	GitHub Actions checks include Jira validation, frontend ESLint, frontend build, Docker build, and Docker Compose validation

- **AI usage Log**

(This table summarizes AI contribution in this document)

Section	Who	AI contribution (%)	Description	Verified by
Testing Summary	Stephanie	35%	I used Claude to help structure and word each subsection based on my meeting notes, code review, and what the team discussed	
Automated Testing	Stephanie	30%	I used Claude to help write up the CI/CD checks based on what I found/understand in the GitHub Actions pipeline.	
Manual Testing	Stephanie	25%	I used Claude to help format the manual test cases into the template and improve wording for expected results, actual results, pass/fail status, and blocked test cases. The actual test results were based on my manual testing.	
Automated Testing	Stephanie	30%	I used Claude to help summarize the frontend Jest tests, backend pytest tests, CI/CD checks, and current testing limitations based on the source code and GitHub Actions pipeline.	
Testing Metrics	Stephanie	20%	I used Claude to help calculate and organize the testing metrics, including total manual tests, passed tests, failed tests, blocked tests, defect count, and automated testing status.	

- **References**

- **Glossary**